docs/handbook/17-USER-MANAGEMENT-USECASES.md

User management in practice — roles, visibility and the daily use-cases

The field companion to 07 — User Management & RBAC. That chapter explains the machinery (the signed context, the Access service, OIDC, tenant isolation); this one answers the questions the desk actually asks: *what can this role do, what can it see, what is refused, and how do I manage the employees?* Every table here is transcribed from the enforced matrices in `packages/evam-backend-core/evam_backend_core/rbac.py` (policy **v3.8**) — the same rows the Access service seeds and every service checks. Nothing in this document is a convention; it is all refused server-side when violated.

The worked examples use the Evam roster as imported (People Master) and provisioned (Access users) in production.

---

1. The model in five sentences

1. There are **twelve roles** in three tiers: Leadership (Admin, Management), Heads (BD / Credit / Syn / AM Head, LMS Management), and ICs (BDRM, Deal Analyst, Syn RM, AM RM, LMS Operator).
2. Every module and operation is a matrix cell holding one of five levels: `-` **none** · `R` **read-only** · `S` **scoped read-write (your own book)** · `F` **full read-write** · `A` **approve** (a decision, not a data write).
3. **Role stacking = max().** A person holding several roles gets the *highest* level any of them grants, cell by cell. This is powerful and easy to over-use — see §7.
4. **Sign-in roles decide permissions; roster roles decide dropdowns.** The Access service's users (Masters → Employees, the rows with sign-in) carry the roles the matrices check. The register's people directory (imported from People Master) feeds the RM/Analyst pickers. They are two directories on purpose — §6.4.
5. Four verbs are **maker–checker**: committee decision, CP checklist approval, loan-account authorization, Advaya handover. The preparer can never be the approver — the register refuses the same person on both sides.

---

2. Who sees what — the views matrix in words

`F`ull · `S`coped (own book) · `R`ead-only · `-` invisible.

Tab	Admin	Mgmt	BD Head	BDRM	Credit Head	Deal Analyst	Syn Head	Syn RM	AM Head	AM RM	LMS Op	LMS Mgmt
Today / Dashboard	F	F	S	S	S	S	S	S	S	S	S	S
Leads	F	F	F	S	—	—	S	S	S	S	—	—
Deals	F	F	F	S	S	S	S	S	S	S	—	—
Lending	F	F	F	S	F	S	R	R	R	R	R	R
Platform Deals (Syn)	F	F	F	S	S	S	F	S	R	R	—	—
Asset Monetisation	F	F	F	S	S	S	R	R	F	S	—	—
FI Master	F	F	F	R	S	S	F	S	R	R	—	—
Clients	F	F	F	S	R	R	S	S	S	S	R	R
Employees	F	F	R	R	R	R	R	R	R	R	—	—
Audit / Activity log	F	—	—	—	—	—	—	—	—	—	—	—
Tools	F	R	R	R	R	R	R	R	R	R	—	—

Reading it:

- **The credit desk cannot see Leads at all** — Credit Head and Deal Analyst have `-` there. A lead is BD's property until it converts.
- **Everyone can read the whole Lending book** (`R` for the RM desks and servicing) — but reading is where it stops; their writes are their own verbs elsewhere.
- **Only Admin sees the Audit and Activity tabs.** Even Management does not — audit is oversight *of* leadership too.
- **Employees is readable by every desk role** (it is the team directory) but writable only by Admin and Management.
- `S` on Today/Dashboard means the numbers themselves are scoped: a BDRM's dashboard counts their book, not the firm's.

---

3. Who does what — the operations, grouped

Levels as above; `A` = may approve/reject, which is not the same as editing.

Leads (BD's funnel). Add/edit/push-to-deals: Admin, Management, BD Head full; BDRM scoped to their own leads. Reassigning a lead is Head-and-above. Nobody outside BD touches a lead.

Deals & clients. Deal profile edits: leadership + BD full, every desk scoped. Changing deal *ownership* is Head-level (BD/Syn/AM Head for their vertical). Client profile/contract/intel/monitoring edits: BD and the RM desks (scoped); **Credit Head and Deal Analyst are read-only on Clients** — the credit desk consumes the client file, it does not maintain it.

Lending (the credit pipeline). Stage changes: Admin, Management, BD Head, Credit Head full; Deal Analyst scoped; **no RM desk moves a lending stage**. Line edits mirror it. Assigning the analyst on any product line is Credit Head / leadership only.

Syndication. The Syn desk's mirror: Syn Head full, Syn RM scoped (lender adds, chases, responses, matrix moves); the credit desk can log chases too (Head full, analyst scoped); AM desk read-only.

Asset monetisation. AM Head full, AM RM scoped; BD scoped; credit desk can edit records (Head full / analyst scoped) since AM mandates ride on deals they govern.

Governance evidence (who may file what). Committee outcomes and sanction letters: credit authority only (Admin / Management / Credit Head). Syndication artefacts: the Syn desk, senior-gated. AM artefacts: the AM desk, senior-gated. Executed documents: whoever may upload documents. An RM can never file a committee outcome.

Maker–checker verbs.

Verb	Maker (prepares)	Checker (approves)
Credit committee decision	BD/credit raise the run	Admin, Management, BD/Credit/Syn/AM Head (`approve_stage_change` = `A`)
CP/CS checklist	Admin, Management, Credit Head, Deal Analyst (scoped)	Admin, Management, Credit Head — **a different person**
Advaya handover / disbursement	Credit Head, Deal Analyst record the package	Admin, Management, Credit Head — a different person
Loan-account booking	Credit Head attests (maker)	**LMS Management only** — origination must not book its own exposure

Employees & administration. `edit_employee` and `add_employee_assign_role`: **Admin and Management only.** `delete_row` anywhere: **Admin only**. Backup/restore: Admin only. Reference data (banks, checklists): leadership + the owning Head.

Everyone signed in can: sign in, see their Today queue, log interactions on their own book, export CSV of what they can already see, snooze their own reminders, and open an EWS case on their own book — a field RM spotting distress is never blocked from raising the flag.

---

4. What `S` (scoped) actually covers

"Your own book" is wider than "rows assigned to you". A row is in scope through any of five doors (`services/register/app/authz/scope.py`):

1. **Assignment** — you are the RM/analyst on that exact line.
2. **Connected company** — you hold an assignment on *any* line of that company, so you can read its whole story (write still needs your own line).
3. **Own book** — rows you created.
4. **Team** — your direct reports' books (a Head sees the team).
5. **Vertical default** — an *unassigned* line belongs to its vertical Head (Lending → Credit Head, Syndication → Syn Head, AM → AM Head). Nothing sits ownerless.

---

5. Role profiles with the Evam roster

What each role is *for*, and the sharpest edges of what it can and cannot do.

Admin — *Vamsi, Kannan, Sunil, Madhusudan, System Administrator, TechAdmin.* The system's operator: everything everywhere, plus the four things nobody else has — the Audit and Activity tabs, hard `delete_row`, backup/restore, and employee/role management (shared with Management). Admin is the break-glass role, not a daily desk; the audit trail records its every touch.

Management — *Sakshi (and stacked on most of the roster — see §7).* Firm-wide oversight: full read-write on every business module, approval authority on every stage-change gate, employee management. Cannot see Audit (Admin-only), cannot hard-delete. This is the strongest business role — treat grants of it accordingly.

BD Head — full command of the origination funnel: every lead, every deal, every push-to-deals, lead reassignment, an approval seat on stage changes. Read-only on Employees; no analyst-assignment authority (that is credit's).

BDRM — *Shubh Dave, Chetan Malik, Pallavi Patel.* The field originator: adds and works **their own** leads, converts them (Hot + approval flow), maintains their clients, uploads documents on their book. Cannot: touch another BDRM's leads, move a lending stage, assign analysts, see Audit, or file governance evidence.

Credit Head — *Pranay Shrivastava.* The credit authority: the whole lending book full, stage changes, analyst assignment on every product line, committee/sanction evidence, CP checklist **checker**, handover maker. Cannot see Leads at all; read-only on Clients; cannot manage employees.

Deal Analyst — *Archana, Bhavana, Prateek, Nirmala (roster desk).* The credit maker on their assigned lines: lending edits and stage moves (scoped), CP checklist **maker**, disbursement request recorder, covenant work (scoped). Cannot: approve what they prepared (four-eyes), see Leads, edit Clients, assign anyone.

Syn Head / Syn RM — *Grishma, Prashant, Ananda (Syn RMs by roster).* The platform-deals desk: mandates, lender matrices, chases (Head full / RM scoped), the syndication evidence trail (Head-gated). Read-only on Lending and AM.

AM Head / AM RM — the mirror for asset monetisation: mandate records, teaser/NDA/ offer evidence, closure approval (Head). Read-only elsewhere.

LMS Operator / LMS Management — the servicing pair (no one holds these yet — Sakshi was mapped to Management instead, deliberately, until servicing goes live). Operator posts routine ledger events; LMS Management alone authorizes loan accounts, classifications, closures. Both read the whole book, neither touches origination rows.

DSA Sanjay — the deliberate exception: exists **only** in the register's people directory (`no sign-in` chip in Employees), role "DSA (external)". His two mandates keep their true owner; no picker offers him for new work; he can never log in. This is the pattern for any external sourcing partner.

6. The employee-management use-cases (stepwise)

All of these are Admin/Management actions; everyone else sees the directory read-only. API calls go to the **Access** service (`/access/v1/...`); the register's people directory syncs from it for the pickers.

6.1 Onboard a new employee

1. Masters → Employees → **Add employee** (or `POST /access/v1/users` with `email`, `full_name`, `short_name`, `is_active`, `roles`).
2. Email is the identity — it must match what Dex/SSO will assert at sign-in, and it is unique per tenant **forever** (even a deactivated user keeps their address).
3. Grant the role that matches the desk (§5), not the widest one that works.
4. First sign-in needs nothing else: the account exists, the roles resolve, the signed context carries them.

6.2 Change someone's roles

- Grant: `POST /access/v1/users/{id}/roles {"role": "..."}` — stacking allowed.
- Revoke: `DELETE /access/v1/users/{id}/roles/{role}`.
- Both bump the user's `permissions_epoch`, so already-issued sessions lose the old authority at the next sensitive operation — no waiting for token expiry.
- **Re-granting a previously revoked role works** (fixed 2026-08-20; earlier builds answered 409 `user_roles_unique` because the revoked grant was soft-deleted and still held the unique slot — the grant now restores it).

6.3 Offboard / deactivate — and why there is no delete

`PATCH /access/v1/users/{id} {"is_active": false}` (or the Employees toggle). There is deliberately **no DELETE**: the audit trail and every `granted_by`, `approved_by`, `updated_by` on record must keep pointing at a real person. A deactivated user cannot sign in, drops out of the default directory listing, and their email stays reserved. To see them again: `GET /access/v1/users?include_inactive=true`. Reactivation is the same PATCH with `true` — a `user.reactivate` audit event and an epoch bump.

6.4 The two directories — why Employees shows "no sign-in"

- **Access users** = accounts: email, roles, active flag. Permissions live here.
- **Register people** = the roster the workbook imports: full names, the handles the trackers store, the RM/Analyst dropdown contents.
- The Employees tab merges them by email. A register person with no access user (DSA Sanjay) shows **no sign-in**; that person can be picked in history but holds no permissions. Creating the access user later joins the two automatically.

6.5 Who approves what — reading a refusal

When the platform refuses with "Role(s) [...] may not perform '...'", the operation name maps to a row in §3. Two frequent ones:

- `approve_stage_change` — the six approval seats (Admin, Management, four Heads).
- `approve_cpcs_checklist` — credit seniors only, and never the preparer.

---

7. A caution from the live roster — role stacking flattens the matrix

The production grants (2026-08-20) stack **Management onto nearly every desk user** (Chetan BDRM+Management, Bhavana Management, Prateek Deal Analyst+Management, …). Because stacking

takes the *maximum* per cell, each of those users now effectively holds the Management column: full read-write on every module, every approval seat, employee management. Two concrete consequences:

- **Bhavana** (currently Management-only in Access, Deal Analyst on the roster) can approve stage changes, edit any BDRM's leads, and add employees — none of which the Deal Analyst desk grants. The scoped/maker limits described in §5 are not in force for her.
- **Four-eyes still holds** (the different-person rule is checked per record, not per role) — but *every* stacked user is now an eligible checker for everything, which weakens the review in practice.

If this is a deliberate trust posture for a small team, it works — the audit trail still names every actor. When the team grows, the intended shape is: each person holds their **desk role** (matching their People Master role), and Management stays with actual leadership. The fix per user is one revoke: `DELETE /access/v1/users/{id}/roles/Management` — and, where the desk role is missing (Archana, Bhavana, Ananda), one grant to add it.

---

8. Widening visibility at runtime — the four levers

Everything in this section is **configuration**: Admin actions through the running platform, no code change, no redeploy. Pick the lever by the shape of the ask.

You want…	Lever	Where
a whole ROLE to see (or edit) a module differently	**matrix override**	`PATCH /access/v1/access`
one PERSON to reach another desk's world	**role stacking**	`POST /access/v1/users/{id}/roles`
several people on the SAME book / company	**assignments**	`POST /v1/assignments` (register)
a senior to see their TEAM's books	**reports_to**	`PATCH /access/v1/users/{id}`

8.1 Matrix override — change what a role means, live

Every cell of §2 and §3 is runtime data (`access_grants`); Admin edits one cell at a time and the change applies tenant-wide to everyone holding that role:

```
PATCH /access/v1/access
{ "kind": "view", "item": "leads", "role": "Deal Analyst", "access": "READ" }
```

Use case: the credit desk keeps asking BD what is coming down the funnel. This one call makes the Leads tab appear for every Deal Analyst — **read-only**: they see the funnel, and every write verb (`add_lead`, `edit_lead`, `push_lead_to_deals`) still answers 403 because the operations cells are untouched. To widen an operation instead, use `"kind": "operation"` — e.g. `{"kind": "operation", "item": "log_chase", "role": "Deal Analyst", "access": "FULL"}` lets analysts log syndication chases beyond their own lines.

Rules of the lever:

- **Four guardrail cells refuse even Admin** (spec hard rules): `delete_row`, `backup_restore`, the `audit` view, the `activity_log` view. Everything else is editable.
- Overrides are tagged `origin: "override"` and audited (`matrix.edit`, with the before/after). `GET /access/v1/drift` reports every divergence from the approved baseline — run it after editing so the change is a decision on record, not a mystery later.

- Levels are the same five words: `NONE`, `READ`, `SCOPED`, `FULL`, `APPROVE`. Granting `APPROVE` on `approve_stage_change` to a role adds an approval seat — that is a governance change; treat it like one.

8.2 Role stacking — widen one person, not the role

If only *Prateek* should see syndication, do not touch the matrix — grant him `Syn RM`. Stacking takes the max per cell, so he gains the Syn RM column (Platform Deals scoped, chases on his own mandates) while every other analyst stays unchanged. This is the per-person lever; the matrix is the per-role lever. §7's caution is the same lever over-used.

8.3 Assignments — several people on the same book

"Belonging to a book" is not a role at all — it is an **assignment row** on a line, and a line or company can carry several at once. The scope doors of §4 then do the work:

Use case — shared coverage of one client: Zeon Electric has a lending line and a syndication mandate. Assign Bhavana (Deal Analyst) on the lending line, Grishma (Syn RM) on the mandate, with Shubh already the BDRM on the deal. Result: **all three read Zeon's entire story** — every line, the timeline, the documents — through the *connected-company* door, while each can **write only the line they hold**. That is "multiple persons on the same book": reads pool, writes stay owned.

```
POST /v1/assignments
{ "user_id": "<bhavana's access-user uuid>",
  "subject_type": "Lending", "subject_id": "<line-uuid>",
  "assignment_role": "Deal Analyst", "note": "covering Zeon with Shubh" }
```

- **Who may assign is itself a matrix** (`ASSIGNMENT_AUTHORITY`): analysts are placed by Credit Head (or leadership); Syn RMs by Syn Head / BD Head; BDRMs on leads and deals by BD Head. An RM cannot self-assign onto a book.
- A second analyst on the SAME line is allowed — coverage during leave is exactly this: assign the substitute, and end the assignment (`POST /v1/assignments/{id}/end`) when the owner returns. Ending keeps the row as history; the register never forgets who held a book.
- The assignee must be a real, active user allowed to hold that assignment role — the register checks with Access before accepting.

8.4 reports_to — team visibility for a senior

Set the reporting line on the Access user: `PATCH /access/v1/users/{prateek-id} {"reports_to": "<pranay-id>"}`. Access resolves the **transitive** tree at sign-in and forwards it; the register's *team* door then puts every report's book inside the senior's scope, with the senior's own access level. *Use case:* Pranay (Credit Head) automatically sees and works every line his analysts hold — no assignment on each line needed. This also feeds the Employees grid's "Reports to" column.

8.5 How fast each takes effect

The gateway caches the identity answer for **60 seconds** (hard cap 5 minutes), so matrix edits, role grants and `reports_to` changes reach every service within about a minute — and role changes bump the user's `permissions_epoch`, which sensitive operations re-check immediately. Assignments are read fresh per request in the register: effective on the next click.

---

9. Where to verify all of this

- The enforced matrices: `packages/evam-backend-core/evam_backend_core/rbac.py` (`VIEW_ACCESS`, `OPERATIONS`) — policy version stamped in every signed context.
- The live, runtime-editable copy: the Access service's `access_grants` table; `GET /access/v1/drift` reports any divergence from the baseline.
- The scope evaluator: `services/register/app/authz/scope.py`.
- The machinery (identity, signing, provisioning API, tenant isolation): 07 — User Management & RBAC.